

Pattern Matching and Language Variants

- Document number: P0095R1
- Date: 2016-05-29
- Reply-to: David Sankel <david@stellarscience.com>
- Audience: Evolution

Abstract

Pattern matching and language-based variants improve code readability, make the language easier to use, and prevent common mishaps. This paper proposes a syntax that extends C++ to provide these two commonly requested features.

```
lvariant json_value {
    std::map< std::string, json_value > object;
    std::vector< json_value > array;
    std::string string;
    double number;
    bool boolean;
    std::monostate nullary;
};
```

History

P0095R1. Merged in blog post developments. Added `nullptr` patterns, `@` patterns, and pattern guards. A mechanism for dealing with assignment was also added. Wording as it relates to patterns was added. Made expression and statement `inspects` use a single keyword.

C++ Langauge Support for Pattern Matching and Variants blog post. Sketched out several ideas on how a more extensive pattern matching feature would look. Discussed an extension mechanism which would allow any type to act tuple-like or variant-like. `lvariant` is used instead of `enum union` based on feedback in Kona.

Kona 2015 Meeting. There was discussion on whether or not a partial pattern-matching solution would be sufficient for incorporation of a language-based variant. While exploration of a partial solution had consensus at 5-12-8-2-0, exploration of a full solution had a strong consensus at 16-6-5-1-0. The question was also asked whether or not we want a language-based variant and the result was 2-19-6-0-1.

P0095R0. The initial version of this paper presented in Kona. It motivated the need for a language-based variant and sketched a basic design for such a feature with the minimal pattern matching required.

Before/After Comparisons

Figure 1. Declaration of a command data structure.

before	after
<pre>struct set_score { std::size_t value; }; struct fire_missile {};; struct fire_laser { unsigned intensity; }; struct rotate { double amount; }; struct command { std::variant< set_score, fire_missile, fire_laser, rotate > value; };</pre>	<pre>lvariant command { std::size_t set_score; std::monostate fire_missile; unsigned fire_laser; double rotate; };</pre>

Figure 2: Implementation of an output operator

before	after
<pre>namespace { struct Output { std::ostream& operator()(std::ostream& stream, const set_score& ss) const { return stream << "Set the score to " << ss.value << ".\n"; } std::ostream& operator()(std::ostream& stream, const fire_missile&) const { return stream << "Fire a missile.\n"; } std::ostream& operator()(std::ostream& stream, const fire_laser& fl) const { return stream << "Fire a laser with " << fl.intensity << " intensity.\n"; } std::ostream& operator()(std::ostream& stream, const rotate& r) const { return stream << "Rotate by " << r.degrees << " degrees.\n"; } };</pre>	<pre>std::ostream& operator<<(std::ostream& stream, const command& cr) { return inspect(cmd) { set_score value => stream << "Set the score to " << value << ".\n" fire_missile _ => stream << "Fire a missile.\n" fire_laser intensity => stream << "Fire a laser with " << intensity << " intensity" rotate degrees => stream << "Rotate by " << degrees << " degrees.\n" } }</pre>

```
}
};
}

std::ostream& operator<<(std::ostream& stream, const command& cmd) {
    return std::visit(std::bind(Output(), std::ref(stream), std::placeholders:
        cmd.value);
}
}
```

Figure 3: Switching an enum.

before	after
<pre>enum color { red, yellow, green, blue };</pre>	
<pre>const Vec3 opengl_color = [&c] { switch(c) { case red: return Vec3(1.0, 0.0, 0.0); case yellow: return Vec3(1.0, 1.0, 0.0); case green: return Vec3(0.0, 1.0, 0.0); case blue: return Vec3(0.0, 0.0, 1.0); default: std::abort(); } };</pre>	<pre>const Vec3 opengl_color = inspect(c) { red => Vec3(1.0, 0.0, 0.0) yellow => Vec3(1.0, 1.0, 0.0) green => Vec3(0.0, 1.0, 0.0) blue => Vec3(0.0, 0.0, 1.0) };</pre>

Figure 4: Expression Datatype

before	after
<pre>struct expression; struct sum_expression { std::unique_ptr<expression> left_hand_side; std::unique_ptr<expression> right_hand_side; }; struct expression { std::variant<sum_expression, int, std::string> value; }; expression simplify(const expression & exp) { if(sum_expression const * const sum = std::get_if<sum_expression>(&exp)) { if(int const * const lhsInt = std::get_if<int>(sum->left_hand_side.get())) && *lhsInt == 0) { return simplify(*sum->right_hand_side); } else if(int const * const rhsInt = std::get_if<int>(sum->right_hand_side.get()) && *rhsInt == 0) { return simplify(*sum->left_hand_side); } else { return {sum_expression{ std::make_unique<expression>(simplify(*sum->left_hand_side)), std::make_unique<expression>(simplify(*sum->right_hand_side))}} } } return exp; } void simplify2(expression & exp) { if(sum_expression * const sum = std::get_if<sum_expression>(&exp)) { if(int * const lhsInt = std::get_if<int>(sum->left_hand_side.get()) && *lhsInt == 0) { expression tmp(std::move(*sum->right_hand_side)); exp = std::move(tmp); simplify(exp); } else if(int * const rhsInt = std::get_if<int>(sum->right_hand_side.get()) && *rhsInt == 0) { expression tmp(std::move(*sum->left_hand_side)); exp = std::move(tmp); simplify(exp); } else { simplify(*sum->left_hand_side); simplify(*sum->right_hand_side); } } return exp; }</pre>	<pre>lvariant expression; struct sum_expression { std::unique_ptr<expression> left_hand_side; std::unique_ptr<expression> right_hand_side; }; lvariant expression { sum_expression sum; int literal; std::string var; }; expression simplify(const expression & exp) { return inspect(exp) { sum {*(literal 0), *rhs} => simplify(rhs) sum {*lhs, *(literal 0)} => simplify(lhs) sum {*lhs, *rhs} => expression::sum{ std::make_unique<expression>(simplify(lhs)), std::make_unique<expression>(simplify(rhs))}; _ => exp }; } void simplify2(expression & exp) { inspect(exp) { sum {*(literal 0), *rhs} => { expression tmp(std::move(rhs)); exp = std::move(tmp); simplify2(exp); } sum {*lhs, *(literal 0)} => { expression tmp(std::move(lhs)); exp = std::move(tmp); simplify2(exp); } sum {*lhs, *rhs} => { simplify2(lhs); simplify2(rhs); } _ => ; }; }</pre>

Figure 4: struct inspection

before	after
--------	-------

<pre> struct player { std::string name; int hitpoints; int lives; }; </pre>	
<pre> void takeDamage(player &p) { if(p.hitpoints == 0 && p.lives == 0) gameOver(); else if(p.hitpoints == 0) { p.hitpoints = 10; p.lives--; } else if(p.hitpoints <= 3) { p.hitpoints--; messageAlmostDead(); } else { p.hitpoints--; } } </pre>	<pre> void takeDamage(player &p) { inspect(p) { {hitpoints: 0, lives:0} => gameOver(); {hitpoints:hp@0, lives:1} => hp=10, 1--; {hitpoints:hp} if (hp <= 3) => { hp--; messageAlmostDead(); } {hitpoints:hp} => hp--; } } </pre>

Introduction

There is general agreement that language-based variants and pattern matching would make C++ programmers more productive. However, a design that has the elegance of a functional language, but with the performance and utility of a systems language is not forthcoming. Add to that the requirements of backwards compatibility and consistency with the rest of C++, and we've got quite a challenging problem.

This paper presents a design for language variants and pattern matching that the authors feel has the right mix of syntactic elegance, low-level performance, and consistency with other language features.

The following snippet illustrates our proposed syntax.¹

```

// This lvariant implements a value representing the various commands
// available in a hypothetical shooter game.
lvariant command {
    std::size_t set_score; // Set the score to the specified value
    std::monostate fire_missile; // Fire a missile
    unsigned fire_laser; // Fire a laser with the specified intensity
    double rotate; // Rotate the ship by the specified degrees.
};

// Output a human readable string corresponding to the specified 'cmd' command
// to the specified 'stream'.
std::ostream& operator<<( std::ostream& stream, const command cmd ) {
    return inspect( cmd ) {
        set_score value =>
            stream << "Set the score to " << value << ".\n"
        fire_missile m =>
            stream << "Fire a missile.\n"
        fire_laser intensity:
            stream << "Fire a laser with " << intensity << " intensity.\n"
        rotate degrees =>
            stream << "Rotate by " << degrees << " degrees.\n"
    };
}

// Create a new command 'cmd' that sets the score to '10'.
command cmd = command::set_score( 10 );

```

Motivation

The current library-based variant proposal solves an important need², but they are too complicated for novice users. We describe difficult corner cases, the pitfalls of using types as tags, and the difficulty of writing portable code using a library based variant. All of these problems suggest the necessity of a language-based variant feature in C++.

The struct/tuple and enumerated-union/variant connection

Basic struct types that have independently varying member variables³ have a close relationship to the `std::tuple` class. Consider the following two types:

```

// point type as a struct
struct point {
    double x;
    double y;
    double z;
};

// point type as a tuple
using point = std::tuple< double, double, double >;

```

It is clear that both `point` types above can represent a 3D mathematical point. The difference between these two types is, essentially, the tag which is used to discriminate between the three elements. In the `struct` case, an identifier is used (x, y, and z), and in the `std::tuple` case, an integer index is used (0, 1, and 2).

Although these two point implementations are more-or-less interchangeable, it is not always preferable to use a `struct` instead of a `std::tuple` nor vice-versa. In particular, we have the following general recommendations:

1. If the type needs to be created on the fly, as in generic code, a `std::tuple` must be used.
2. If an integer index isn't a clear enough identifier, a `struct` should be used.
3. Arguably, if inner types aren't essentially connected or if the structure is used only as the result of a function and is immediately used, a `std::tuple` is preferable.
4. In general, prefer to use a `struct` for improved code clarity.

Some may argue that through use of `std::get`, which allows one to fetch a member of a tuple by type, one can achieve all the benefits of a `struct` by using a tuple instead. To take advantage of this feature, one needs to ensure that each inner type has its own distinct type. This can be accomplished through use of a wrapper. For example:

```
struct x { double value; };
struct y { double value; };
struct z { double value; };

using point = std::tuple< x, y, z >;
```

Now one could use `std::get<x>` to fetch the 'x' value of the tuple, `std::get<y>` for 'y' and so on.

Should we use this approach everywhere and deprecate the use of `struct` in any context? In the author's opinion we should not. The use of wrapper types is much more complicated to both read and understand than a plain `struct`. For example, the wrapper types that were introduced, such as the 'x' type, make little sense outside of their corresponding tuples, yet they are peers to it in scope. Also, the heavy syntax makes it difficult to understand exactly what is intended by this code.

What does all this have to do with enumerated unions? The enumerated union is to `std::variant` as `struct` is to `std::tuple`. A variant type that represents a distance in an x direction, a y direction, *or* a z direction (mathematically called a "copoint") has a similar look and feel to the `std::tuple` version of `point`.

```
struct x { double value; };
struct y { double value; };
struct z { double value; };

using copoint = std::variant< x, y, z >;
```

This copoint implementation has the same drawbacks that the `std::tuple` implementation of points has. An enumerated union version of `copoint`, on the other hand, is easier to grok and doesn't require special tag types at all.

```
lvariant copoint {
    double x;
    double y;
    double z;
};
```

SFINE in basic usage

Some variation of the following example is common when illustrating a `std::variant` type:

```
void f( std::variant< double, std::string> v ) {
    if( std::holds_alternative< double >( v ) ) {
        std::cout << "Got a double " << std::get< double >( v ) << std::endl;
    }
    else {
        std::cout << "Got a string " << std::get< std::string >( v ) << std::endl;
    }
};
```

This illustrates how quickly variants can be disassembled when they are simple, but it is hardly representative of how complex variant types are used. The primary problem in the above snippet is that there are no compile-time guarantees that ensure all of the *n* alternatives are covered. For the more general scenario, a `visit` function is provided.⁴

```
struct f_visitor {
    void operator()( const double d ) {
        std::cout << "Got a double " << d << std::endl;
    }
    void operator()( const std::string & s ) {
        std::cout << "Got a string " << s << std::endl;
    }
};

void f( std::variant< double, std::string > v ) {
    std::visit( f_visitor(), v );
};
```

Aside from the unsightly verbosity of the above code, the mechanism by which this works makes the visitor's `operator()` rules work by SFINE, which is a significant developer complication. Using a template parameter as part of a catch-all clause is going to necessarily produce strange error messages.

```
struct f_visitor {
    template< typename T >
    void operator()( const T & t ) {
```

```

        // oops
        std::cout << "I got something " << t.size() << std::endl;
    }
};

void f( std::variant< double, std::string > v ) {
    // Unhelpful error message awaits. Erroneous line won't be pointed out.
    std::visit( f_visitor(), v );
};

```

While the utility of type selection and SFINE for visitors is quite clear for advanced C++ developers, it presents significant hurdles for the beginning or even intermediate developer. This is especially true when it is considered that the `visit` function is the only way to guarantee a compilation error when all cases are not considered.

Duplicated types: switching on the numeric index

Using types as accessors with a `std::variant` works for many use cases, but not all. If there is a repeated type the only options are to either use wrapper types or to work with the real underlying discriminator, an integer index. To illustrate the problems with using the index, consider the following implementation of `copoint`:

```

using copoint = std::variant< double, double, double >;

```

Use of both `std::get<double>` and the standard `std::visit` are impossible due to the repeated `double` type in the variant. Using the numeric index to work around the issue brings its own problems, however. Consider the following visitor:

```

struct visit_f {
    void operator()( std::integral_constant<std::size_t, 0>, double d ) {
        std::cout << d << " in x" << std::endl;
    };
    void operator()( std::integral_constant<std::size_t, 1>, double d ) {
        std::cout << d << " in y" << std::endl;
    };
    void operator()( std::integral_constant<std::size_t, 2>, double d ) {
        std::cout << d << " in z" << std::endl;
    };
};

```

Here we introduce yet another advanced C++ feature, compile-time integrals. In the opinion of the author, this is unfriendly to novices. The problem of duplicated types can be even more insidious, however...

Portability problems

Consider the following code:

```

using json_integral = std::variant< int, unsigned, std::size_t, std::ptrdiff_t >;

```

On most platforms, this code will compile and run without a problem. However, if `std::size_t` happens to be typedef'd to be the same type as `unsigned` on a particular platform, a compilation error will ensue. The only two options for fixing the error are to fall back to using the index or to make custom wrapper types.

Also notable is that working with third party libraries that are free to change their underlying types creates abstraction leaks when used with a library-based variant.

```

// Is this code future proof? Not likely. Looks like a foot-gun to me.
using database_handle = std::variant< ORACLE_HANDLE, BERKELEY_HANDLE >;

```

Because enumerated unions require identifiers as tags, they aren't susceptible to this problem:

```

lvariant database_handle {
    ORACLE_HANDLE oracle;
    BERKELEY_HANDLE berkeley;
};

```

Language Based Variant lvariant

The definition of an `lvariant` has the same syntax as a union, but with an `lvariant` keyword as in the following example:

```

// This lvariant implements a value representing the various commands
// available in a hypothetical shooter game.
lvariant command {
    std::size_t set_score; // Set the score to the specified value
    std::monostate fire_missile; // Fire a missile
    unsigned fire_laser; // Fire a laser with the specified intensity
    double rotate; // Rotate the ship by the specified degrees.
};

```

Each member declaration consists of a type followed by its corresponding identifier.

Construction and Assignment

An lvariant has a default constructor if its first field also has a default constructor. A default constructed lvariant is set to the first fields's default constructed value.

Assignment at construction can be used to set the lvariant to a particular value. The lvariant is used as a namespace when specifying specific alternatives.

```
command cmd = command::set_score( 10 );
```

lvariant instances can also be assigned in the course of a program's execution.

```
cmd = command::fire_missile( );
```

Inspection

Extracting values from an lvariant is accomplished with a new `inspect` keyword. Much more will be said about this keyword in the pattern matching section of this paper.

```
inspect( cmd ) {
    set_score value =>
        stream << "Set the score to " << value << ".\n";
    fire_missile m =>
        stream << "Fire a missile.\n";
    fire_laser intensity =>
        stream << "Fire a laser with " << intensity << " intensity.\n";
    rotate degrees =>
        stream << "Rotate by " << degrees << " degrees.\n";
}
```

Assignment

As with library-based variants, the behavior of assignment when an exception is thrown is of considerable concern. We propose the following for lvariants:

- If any of the alternatives is not friendly (ie. has a possibly throwing move constructor or a possibly throwing move assignment operator), there will not be a default assignment operator for the lvariant.
- Users will have the ability to implement their own assignment operator to their liking.

This provides a safe default and supports users of differing philosophies.

The "I'm broken. You deal with it." philosophy allows the lvariant to get into a state where the only valid operations are assignment and destruction. This is accomplished by overriding the assignment operator and allowing the 'std::valueless_by_exception' exception to pass through to callers.

```
lvariant Foo {
    PossiblyThrowingMoveAssignmentType field1;
    std::string field2;

    // Possibly throw a 'std::valueless_by_exception' exception which makes this
    // object only assignable and destructable.
    Foo& operator=(const Foo& rhs);
    Foo& operator=(const Foo&& rhs); // implementation skipped
};

Foo& Foo::operator=(const Foo& rhs)
{
    // This can possibly throw a 'std::valueless_by_exception' exception.
    lvariant(*this) = rhs;
}
```

The "exception are for sissies" philosophy essentially terminates the program if there's an exception on assignment. This is accomplished by marking the assignment operator `noexcept`.

```
lvariant Foo {
    PossiblyThrowingMoveAssignmentType field1;
    std::string field2;

    Foo& operator=(const Foo& rhs) noexcept;
    Foo& operator=(const Foo&& rhs) noexcept; // implementation skipped
};

Foo& operator=(const Foo& rhs) noexcept
{
    lvariant(*this) = rhs;
}
```

The "embrace emptiness" philosophy switches to a special empty state if there's an exception on assignment. This is accomplished by handling the `std::valueless_by_exception` exception within the assignment operator.

```
lvariant Foo {
    PossiblyThrowingMoveAssignmentType field1;
    std::string field2;
```

```
std::monostate empty;

Foo& operator=(const Foo& rhs);
Foo& operator=(const Foo&& rhs); // implementation skipped
};

Foo& operator=(const Foo& rhs)
{
    try {
        lvariant(*this) = rhs;
    }
    catch(std::valueless_by_exception&) {
        lvariant(*this) = Foo::empty();
    }
}
```

Pattern Matching With inspect

Pattern matching goes far beyond `lvariants`. This section overviews the proposed pattern matching syntax and how it applies to all types.

Lets define some useful terms for discussing pattern matching and variants in C++. We use the word "piece" to denote a field in a `struct`. The word "alternative" is used for `lvariant` fields. The programming language theory savvy will also recognize `lvariants` to be [sum types](#) and simple `structs` to be [product types](#), although we won't use that jargon here.

Pattern matching integrals and enums

The most basic pattern matching is that of integral (ie. `int`, `long`, `char`, etc.) and `enum` types, and that is the subject of this section. Before we get there, however, we need to distinguish between the two places pattern matching can occur. The first is in the statement context. This context is most useful when the intent of the pattern is to produce some kind of action. The `if` statement, for example, is used in this way. The second place pattern matching can occur is in an expression context. Here the intent of the pattern is to produce a value of some sort. The trinary operator `?:`, for example, is used in this context. Upcoming examples will help clarify the distinction.

The context is distinguished by whether or not the cases consist of a statement (ends in a semicolon or is wrapped in curly braces) or an expression.

In the following example, we're using `inspect` as a statement to check for certain values of an `int i`:

```
inspect(i) {
    0 =>
        std::cout << "I can't say I'm positive or negative on this syntax."
                  << std::endl;

    6 =>
        std::cout << "Perfect!" << std::endl;

    _ =>
        std::cout << "I don't know what to do with this." << std::endl;
}
```

The `_` character is the pattern which always succeeds. It represents a wildcard or fallthrough. The above code is equivalent to the following `switch` statement.

```
switch(i) {
    case 0:
        std::cout << "I can't say I'm positive or negative on this syntax."
                  << std::endl;

        break;
    case 6:
        std::cout << "Perfect!" << std::endl;
        break;
    default:
        std::cout << "I don't know what to do with this." << std::endl;
}
```

`inspect` can be used to pattern match within expression contexts as in the following example. `c` is an instance of the `color` enum:

```
enum color { red, yellow, green, blue };

// elsewhere...

const Vec3 opengl_color = inspect(c) {
    red    => Vec3(1.0, 0.0, 0.0)
    yellow => Vec3(1.0, 1.0, 0.0)
    green  => Vec3(0.0, 1.0, 0.0)
    blue   => Vec3(0.0, 0.0, 1.0)
};
```

Note that the cases do not end in a semicolon.

It is also important to note that if an `inspect` expression does not have a matching pattern, an `std::no_match` exception is thrown. This differs from `inspect` statements which simply move on to the next statement if no pattern matches.

All we've seen so far is a condensed and safer `switch` syntax which can also be used in expressions. Pattern matching's real power comes when we use more complex patterns. We'll see some of that below.

Pattern matching structs

Pattern matching structs in isolation isn't all that interesting: they merely bind new identifiers to each of the fields.

```
struct player {
    std::string name;
    int hitpoints;
    int coins;
};

void log_player( const player & p ) {
    inspect(p) {
        {n,h,c}
        => std::cout << n << " has " << h << " hitpoints and " << c << " coins.";
    }
}
```

n, h, and c are "bound" to their underlying values in a similar way to structured bindings. See [P0217R1](#) for more information on what it means to bind a value.

struct patterns aren't limited to binding new identifiers though. We can instead use a nested pattern as in the following example.

```
void get_hint( const player & p ) {
    inspect( p ) {
        {_, 1, _} => std::cout << "You're almost destroyed. Give up!" << std::endl;
        {_,10,10} => std::cout << "I need the hints from you!" << std::endl;
        {_, _,10} => std::cout << "Get more hitpoints!" << std::endl;
        {_,10, _} => std::cout << "Get more ammo!" << std::endl;
        {n, _, _} => if( n != "The Bruce Dickenson" )
            std::cout << "Get more hitpoints and ammo!" << std::endl;
            else
            std::cout << "More cowbell!" << std::endl;
    }
}
```

While the above code is certainly condensed, it lacks clarity. It is tedious to remember the ordering of a struct's fields. Not all is lost, though; Alternatively we can match using field names.

```
void get_hint( const player & p ) {
    inspect(p) {

        {hitpoints:1}
        => std::cout << "You're almost destroyed. Give up!" << std::endl;

        {hitpoints:10, coins:10}
        => std::cout << "I need the hints from you!" << std::endl;

        {coins:10}
        => std::cout << "Get more hitpoints!" << std::endl;

        {hitpoints:10}
        => std::cout << "Get more ammo!" << std::endl;

        {name:n}
        => if( n != "The Bruce Dickenson" )
            std::cout << "Get more hitpoints and ammo!" << std::endl;
            else
            std::cout << "More cowbell!" << std::endl;
    }
}
```

Finally, our patterns can incorporate guards through use of an if clause. The last pattern in the above function can be replaced with the following two patterns:

```
{name:n} if( n == "The Bruce Dickenson" ) => std::cout << "More cowbell!" << std::endl;
_ => std::cout << "Get more hitpoints and ammo!" << std::endl;
```

Pattern matching lvariants

Pattern matching is the easiest way to work with lvariants. Consider the following binary tree with int leaves.

```
lvariant tree {
    int leaf;
    std::pair< std::unique_ptr<tree>, std::unique_ptr<tree> > branch;
}
```

Say we need to write a function which returns the sum of a tree object's leaf values. Variant patterns are just what we need. A pattern which matches an alternative consists of the alternative's name followed by a pattern for its associated value.

```
int sum_of_leaves( const tree & t ) {
    return inspect( t ) {
        leaf i => i
        branch b => sum_of_leaves(*b.first) + sum_of_leaves(*b.second)
    };
}
```

Assuming we can pattern match on the `std::pair` type, which we'll discuss later, this could be rewritten as follows.

```
int sum_of_leaves( const tree & t ) {
    return inspect( t ) {
        leaf i => i
        branch {left, right} => sum_of_leaves(*left) + sum_of_leaves(*right)
    };
}
```

More complex datatypes

Pattern matching can make difficult code more readable and maintainable. This is especially true with complex patterns. Consider the following arithmetic expression datatype:

```
// An lvariant (forward) declaration.
lvariant expression;

struct sum_expression {
    std::unique_ptr<expression> left_hand_side;
    std::unique_ptr<expression> right_hand_side;
};

lvariant expression {
    sum_expression sum;
    int literal;
    std::string var;
};
```

We'd like to write a function which simplifies expressions by exploiting $\text{exp} + 0 = 0$ and $0 + \text{exp} = 0$ identities. Here is how that function can be written with pattern matching.

```
// The behavior is undefined unless `exp` has no null pointers.
expression simplify( const expression & exp ) {
    return inspect( exp ) {
        sum {*(literal 0),      *rhs} => simplify(rhs)
        sum {*lhs              , *(literal 0)} => simplify(lhs)
        _ => exp
    };
}
```

Here we've introduced a new `*` keyword into our patterns. `*<pattern>` matches against types which have a valid dereferencing operator and uses `<pattern>` on the value pointed to (as opposed to matching on the pointer itself). A special dereferencing pattern syntax may seem strange for folks coming from a functional language. However, when we take into account that C++ uses pointers for all recursive structures it makes a lot of sense. Without it, the above pattern would be much more complicated.

Opting into Pattern Matching with Custom Types

Pattern matching tuple-like types

Now we have patterns for integrals, enums, simple structs, and lvariants. Is there a way to enable pattern matching for custom data types? The answer, of course, is yes.

Tuple-like types are those which behave a lot like simple structs. These objects represent a sequence of values of various types. `std::pair` and `std::tuple` are notable examples. In this section we'll see how we can annotate custom tuple types for pattern matching.

Pattern matching for tuple-like types is accomplished by overloading the `extract` operator. Imagine we have a custom `pair` type that has its `m_first` and `m_second` member variables declared private. We overload the `extract` operator as follows:

```
template <class T1, class T2>
class pair {
    T1 m_first;
    T2 m_second;

public:
    // etc.

    operator extract( std::tuple_piece<T1> x, std::tuple_piece<T2> y ) {
        x.set( &this->first );
        y.set( &this->second );
    }
};
```

The signature of the `extract` operator function provides both the number of pieces and the type of each piece. The code in the body of this operator overload connects the actual pieces `m_first` and `m_second` to their placeholders `x` and `y`. This is all that is required for the compiler to use tuple-like objects in pattern matching.

```
inspect(pair<int,std::string>(3, "Hello World")) {
    {3, s} => std::cout << "Three, a special number, says " << s << std::endl;
    {i, s} => std::cout << i << ", a boring number, says " << s << std::endl;
}
```

It is also possible to incorporate named labels. We could do this by setting the second template parameter of the `std::tuple_piece` type:

```
template <class T1, class T2>
struct pair {
    enum field_name { first, second };

    operator extract( std::tuple_piece<T1, first> x, std::tuple_piece<T2, second> y ) {
        x.set( &this->m_first );
        y.set( &this->m_second );
    }
private:
    T1 m_first;
    T2 m_second;
};
```

Then we could do:

```
inspect(pair<double,int>(1.0, 3)) {
    {first:d, second:3} => std::cout << "The " << d << " double goes with the 3 int" << std::endl;
}
```

Pattern matching variant-like types

We would also like to generalize matching for variant-like types. Our example is an `either` template. It is the variant analogue to `std::pair`.

```
template <class T1, class T2>
lvariant either {
    T1 left;
    T2 right;
};
```

Of course, the above implementation will pattern match without modification since we are using an `lvariant`. Let us consider, for the sake of discussion, that the data type was implemented as follows:

```
template<typename T, typename U>
class either
{
public:
    enum selection { left, right };
private:
    selection m_selection;
    T m_left;
    U m_right;
public:
    either( ) : m_selection( left ) {}

    either( T t ) : m_selection( left ), m_left( t ) {}

    either( U u ) : m_selection( right ), m_right( u ) {}

    bool selection() const { return m_selection; }

    T& get_left() {
        assert( m_selection == left );
        return m_left;
    }

    U& get_right() {
        assert( m_selection == right );
        return m_right;
    }

    either<T,U>& operator=( const either<T,U>& other ) {
        m_selection = other.m_selection;
        m_left = other.m_left;
        m_right = other.m_right;
    }

    either<T,U>& operator=( const T& t ) {
        m_selection = left;
        m_left = t;
    }

    either<T,U>& operator=( const U& u ) {
        m_selection = right;
        m_right = u;
    }
};
```

To enable pattern matching for this type, we need to implement two operator overloads: `discriminator` and `alternative`.

```
template<typename T, typename U>
class either {
    //...

public:
    //...
```

```

selection operator discriminator() {
    return m_selection;
}

operator alternative( std::variant_piece<T, left> x ) {
    x.set(&m_left);
}

operator alternative( std::variant_piece<U, right> x ) {
    x.set(&m_right);
}
};

```

The discriminator operator overload returns an integral or enum value corresponding to the question of which alternative is currently active.

The alternative operator is an overloaded function taking in a single `std::variant_piece` parameter. The first template argument of `std::variant_piece` is the type of that alternative. The second template argument is a value of the return type of discriminator [5](#).

Now we have enough information to use our specialized either class in pattern matching:

```

either<std::string, int> e = /* etc. */;

inspect(e) {
    left error_string
        => std::cout << "You've got an error: " << error_string << std::endl;
    right i
        => std::cout << "You've got the answer " << i << std::endl;
}

```

Note that `left` and `right` were not qualified as in `either::left` and `either::right`. The intent in our design is that discriminators in class scope are directly available from within `inspect` statements.

Wording Skeleton

What follows is an incomplete wording for inspection presented for the sake of discussion.

Inspect Statement

inspect-statement:
inspect (expression) { inspect-statement-cases_{opt} }

inspect-statement-cases:
inspect-statement-case inspect-statement-cases_{opt}

inspect-statement-case:
guarded-inspect-pattern => statement

The identifiers in *inspect-pattern* are available in *statement*.

In the case that none of the patterns match the value, execution continues.

Inspect Expression

inspect-expression:
inspect (expression) { inspect-expression-cases_{opt} }

inspect-expression-cases:
inspect-expression-case inspect-expression-cases_{opt}

inspect-expression-case:
guarded-inspect-pattern => expression

The identifiers in *inspect-pattern* are available in *expression*.

In the case that none of the patterns match the value, a `std::no_match` exception is thrown.

Inspect Pattern

guarded-inspect-pattern:
inspect-pattern guard_{opt}

guard:
if (condition)

inspect-pattern:
 –
 nullptr
 * *inspect-pattern*
 (*inspect-pattern*)
 identifier (@ (*inspect-pattern*))_{opt}

alternative-selector inspect-pattern constant-expression { tuple-like-patterns_{opt} }

Wildcard pattern

inspect-pattern:
—

The wildcard pattern matches any value and always succeeds.

nullptr pattern

inspect-pattern:
 nullptr

The nullptr pattern matches values *v* where *v* == nullptr.

Dereference pattern

inspect-pattern:
 * *inspect-pattern*

The dereferencing pattern matches values *v* where *v* != nullptr and where **v* matches the nested pattern.

Parenthesis pattern

inspect-pattern:
 (*inspect-pattern*)

The dereferencing pattern matches *inspect-pattern* and exists for disambiguation.

Binding pattern

inspect-pattern:
 identifier (@ (*inspect-pattern*))_{opt}

If @ is not used, the binding pattern matches all values and binds the specified identifier to the value being matched. If @ is used, the pattern is matched only if the nested pattern matches the value being matched.

Alternative pattern

inspect-pattern:
 alternative-selector inspect-pattern

alternative-selector:
 constant-expression
 identifier

The alternative pattern matches against lvariant values and objects which overload the discriminator and alternative operators. The pattern matches if the value has the appropriate discriminator value and the nested pattern matches the selected alternative.

The *constant-expression* shall be a converted constant expression (5.20) of the type of the inspect condition's discriminator. The *identifier* will correspond to a field name if inspect's condition is an lvariant or an identifier that is within scope of the class definition opting into the alternative pattern.

Integral-enum pattern

inspect-pattern:
 constant-expression

The integral-enum pattern matches against integral and enum types. The pattern is valid if the matched type is the same as the *constant-expression* type. The pattern matches if the matched value is the same as the *constant-expression* value.

Tuple-like patterns

inspect-pattern:
 { *tuple-like-patterns_{opt}* }

tuple-like-patterns:
 sequenced-patterns
 field-patterns

sequenced-patterns:
 inspect-pattern (, *sequenced-patterns*)_{opt}

field-patterns:
 field-pattern (, *field-patterns*)_{opt}

field-pattern:
 piece-selector : *inspect-pattern*

piece-selector:
 constant-expression
 identifier

Tuple-like patterns come in two varieties: a sequence of patterns and field patterns.

A sequenced pattern is valid if the following conditions are true:

1. The matched type is either a `class` with all public member variables or has a valid extract operator. Say the number of variables or arguments to extract is n .
2. There are exactly n patterns in the sequence.
3. Each of the sequenced patterns is valid for the corresponding piece in the matched value.

A field pattern is valid if the following conditions are true: 1. The matched type is either a `class` with all public member variables or has a valid extract operator. 2. *piece-selectors*, if they are *constant-expression*, must have the same type as the extract operator's `std::tuple_pieces` second template argument. 3. *piece-selectors*, if they are *identifiers*, must correspond to field names in the `class` with all public member variables. 4. Each of the field patterns is valid for the the corresponding piece in the matched value.

Both patterns match if the pattern for each piece matches its corresponding piece.

The *constant-expression* shall be a converted constant expression (5.20) of the type of the inspect condition's extract piece discriminator. The *identifier* will correspond to a field name if inspect's condition is an class or an identifier that is within scope of the class definition opting into the tuple-like pattern.

Design Choices

inspect as a statement and an expression

If inspect were a statement-only, it could be used in expressions via. a lambda function. For example:

```
const Vec3 opengl_color = [&c]{
    inspect(c) {
        red    => return Vec3(1.0, 0.0, 0.0)
        yellow => return Vec3(1.0, 1.0, 0.0)
        green  => return Vec3(0.0, 1.0, 0.0)
        blue   => return Vec3(0.0, 0.0, 1.0)
    } }();
```

Because we expect that inspect expressions will be the most common use case, we feel the syntactic overhead and tie-in to another complex feature (lambdas) too much to ask from users.

inspect with multiple arguments

It is a straightforward extension of the above syntax to allow for inspecting multiple values at the same time.

```
lvariant tree {
    int leaf;
    std::pair< std::unique_ptr<tree>, std::unique_ptr<tree> > branch;
}

bool sameStructure(const tree& lhs, const tree& rhs) {
    return inspect(lhs, rhs) {
        {leaf _, leaf _} => true
        {branch {*lhs_left, *lhs_right}, branch {*rhs_left, *rhs_right}}
            => sameStructure(lhs_left , rhs_left)
                && sameStructure(lhs_right, rhs_right)
        _ => false
    };
}
```

It is our intent that the final wording will allow for such constructions.

Special operator extension mechanism

The committee has discussed several mechanisms that enable user-defined tuple-like types to opt-in to language features. This is discussed at length in P0326R0 and P0327R0. We present the extract, discriminator, and alternative operators as one such option, but we fully expect that only one mechanism should be ultimately available in the standard.

[] or {} for tuple-like access

We use curly braces to extract pieces from tuple-like objects because it closely resembles curly brace initialization of tuple-like objects. There has been some discussion as to whether square brackets are a more appropriate choice for structured binding due to ambiguity issues.

Although our preference is curly braces, we believe that whatever is ultimately decided for structured binding should be mimicked here for consistency.

Conclusion

We conclude that types-as-tags are for astronauts, but variants are for everyone. None of the library implementations thus far proposed are easy enough to be used by beginners; a language feature is necessary. In the author's opinion a library-based variant should complement a language-based variant, but not replace it. And with language-based variants comes pattern matching, another highly desirable feature in the language.

Acknowledgements

Thanks to Vicente Botet Escribá, John Skaller, Dave Abrahams, Bjarne Stroustrup, Bengt Gustafsson, and the C++ committee as a whole for productive design discussions. Also, Yuriy Solodkyy, Gabriel Dos Reis, and Bjarne Stroustrup's prior research into generalized pattern matching as a C++ library has been very helpful.

References

- V. Botet Escribá. Product types access. P0327R0. WG21
- V. Botet Escribá. Structured binding: customization points issues. P0326R0. WG21
- A. Naumann. Variant: a type-safe union for C++17 (v7). [P088R2](#). WG21.
- D. Sankel. [C++ Language Support for Pattern Matching and Variants](#). [davidsankel.com](#).
- Y. Solodkyy, G. Dos Reis, B. Stroustrup. [Open Pattern Matching for C++](#). GPCE 2013.
- H. Sutter, B. Stroustrup, G. Dos Reis. Structured bindings. [P0144R2](#). WG21.

1. The `std::monostate` type, which has only one value, is from variant proposal P088R2.[↵](#)

2. Variant: a type-safe union for C++17 (v7). [P0088R2](#)[↵](#)

3. See [The C++ Core Guidelines](#) rule C.2.[↵](#)

4. Compare that code to the same for an lvariant:

```
lvariant double_or_string {
    double with_double;
    std::string with_string;
};

void f( double_or_string v ) {
    switch( v ) {
        case with_double d:
            std::cout << "Got a double " << d << std::endl;
        case with_string s:
            std::cout << "Got a string " << s << std::endl;
    }
}
```

[↵](#)

5. This syntax will only work if [P0127R1](#) goes into the language, which seems likely. Otherwise, we would need to explicitly specify the discriminator type as in `std::variant_piece<T, selection, left>`.[↵](#)